Energy-Efficient Quadruped Locomotion Based on Deep Reinforcement Learning

Felipe Mohr Santos
Robotics Departament
Senai Cimatec
Salvador, Brazil
ORCID: 0009-0009-2713-829X

Anusio Menezes Correia
Software Department
Robotics Departament
Salvador, Brazil
ORCID: 0000-0002-6479-0705

Erick G. Sperandio Nascimento
Strictu Sensu Department
Senai Cimatec
Salvador, Brazil
ORCID: 0000-0003-2219-0290

Oberdan Rocha Pinheiro
Software Department
Senai Cimatec
Salvador, Brazil
ORCID: 0000-0002-8904-520X

Alex Álisson Bandeira Santos
Strictu Sensu Department
Senai Cimatec
Salvador, Brazil
ORCID: 0000-0001-9935-4084

2025 Brazilian Conference on Robotics (CROS) | 979-8-3315-5288-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/CROS66186.2025.11066113

***Abstract*—Legged robots, especially quadruped robots, have proven to be very useful for different applications due to their ability to locomote over different terrains. However, this type of robot requires significantly more energy compared to other types of simpler robots. In this work, we implement an energy-efficient locomotion controller based on Deep Reinforcement Learning to follow a desired velocity command minimizing power consumption, and for this we trained a Proximal Policy Optimization (PPO) agent in Isaac Lab simulator, and all the modeling of the environment for training and controlling the robot is presented. We also implement a controller based on Central Pattern Generator so we can compare the power consumption of both controllers. We found that our trained policy learned an efficient gait for locomotion and can spend up to 55.8% less energy compared to the CPG controller in the best case.**

***Index Terms*—quadruped robot, reinforcement learning, energy efficiency, power consumption, central pattern generator**

I. Introduction

LEGGED robots stand out from the rest due to their ability to move around complex terrains, and have been extensively studied and applied in different areas in the last years. Among other legged robots, quadruped robots present a balance between stability and complexity [1], leading to better efficiency and applicability. Even so, due the the high number of electrical motors required for this type of robot, the energy consumption is significantly higher when compared to wheeled or tracked robots [2], limiting its practical application in situations that require a lot of operating time [3].

The use of Reinforcement Learning to control this type of robot is also something that proves to be very promisor and has been widely used in the last years, giving quadruped robots the ability to locomote over different types of terrain without needing a complex control system.

979-8-3315-5288-6/25/$31.00©2025 IEEE

A. Contributions

In this work, we implement an energy-efficient locomotion control in a quadruped robot using Deep Reinforcement Learning (DRL) by implementing a specific reward function that leads to spend less energy, and analyze the results focusing on power consumption. To check if the trained policy can spend less energy, we also implement a controller based on Central Pattern Generator (CPG) [4], a commonly used algorithm for legged robot locomotion, and realize a series of experiments with different parameters and velocities, so we can compare the power consumption of both controllers.

This work is aligned with the United Nations Sustainable Development Goals (UN SDGs) [5], specifically on SDG 7 (Affordable and Clean Energy) and SDG 12 (Responsible Consumption and Production), as it deals directly with energy efficiency in quadruped robots and seeks a form of locomotion that requires less consumption.

The rest of this paper is organized as follows: section II presents related works about reinforcement learning for quadruped locomotion and energy efficiency and section III explains the main concepts required and used in this work. In section IV the development is presented, focusing on the modeling of the Reinforcement Learning environment, the framework used for training and controlling the robot, and the implementation of the CPG-based controller. Section V shows the obtained results after training the agent and comparisons between this agent and the CPG-based controller, focusing on energy efficiency and power consumption. Finally, section VI brings the conclusions of the work.

II. Related Work

Many works deal with the use of DRL for quadruped robot locomotion. Among them, different approaches are adopted, like the use of Reinforcement Learning to dynamically change the parameters of a Model Predictice Control (MPC) controller [6] or a CPG controller [4], generate foot trajectories [7],

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 07,2026 at 09:24:23 UTC from IEEE Xplore. Restrictions apply.

control de joint position of the articulations [8], [9], or even to directly control the torque of the joints [10].

Aractingi et. al uses a reward function that considers the losses by Joule effect and by friction, to minimize the overall power consumption of the Solo12 robot [9] and compares the use of a power penalty function to torque, velocity and acceleration penalties, founding that the first one is easiest to tune and leads to significant reductions on power consumption.

Fu et. al trained three policies to follow different velocity commands [11]. Their work shows that certain gaits are only efficient at certain speeds, and it can be suboptimal to use them for other speeds. Each trained policy converged to the optimal gait at each of the speeds (walk for the lowest speed, trot for the medium and bounce for the greater). For all three gaits, the policies spent less energy than an MPC controlling the robot at the same velocity.

The Cyber Dog robot was trained with DDPG algorithm by Yan et. al to optimize energy consumption [3], by optimizing the plantar trajectories, reducing the energy consumption by 7-9%. Bellegarda and Ijspeert trained a policy for Unitree A1 robot to locomote using CPG [4], whose parameters are defined by the agent. This work does not focus on energy consumption, but minimizes the work, which leads to more efficient gaits. A similar approach is done in [12] and [13].

III. Background

For this work, some main concepts are fundamental: the Reinforcement Learning and Markov Decision Process definitions, the Actor-critic algorithms used for training the agent and the Central Pattern Generators, used to implement a controller to compare with the agent performances.

A. Reinforcement Learning

The Reinforcement Learning problem can be modeled as a Markov Decision Process (MDP) [14].

In MDP, the agent interacts with the environment in a sequence of discrete time steps t, by taking an action $a_t \in \mathcal{A}$, given the current states $s_t \in \mathcal{S}$, and receiving a numerical reward $r_{t+1} \in \mathcal{R} \subset \mathbb{R}$ in the next time step. The MDP can then be denoted as a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$, where $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}_+$ is the transition probability distribution, and the dynamics of the MDP is defined as the probability p that the state in the next time step s_{t+1} will be s', based on the current state s_t and the current action a_t:

$$p(s', s, a) = \Pr(s_{t+1} = s' | s_t = s, a_t = a) \tag{1}$$

The goal of Reinforcement Learning is to find a policy $\pi(s, a) = Pr(a = a|s = s)$ that maximizes the future rewards [15]. This policy represents the probability of taking action a, given the state s, and the optimal policy π^* for an optimal action a^* can be defined as:

$$\pi^*(s, a^*) = \text{argmax}_a \mathbb{E}(\sum_{t=0} \gamma^t r_t) \tag{2}$$

where $\mathbb{E}$ is the expected future reward, r_t represents the reward value obtained at time t and $\gamma \in (0, 1)$ is the discount factor.

B. Actor-Critic Algorithms

In Deep Reinforcement Learning (DRL), the policy $\pi(s, a)$ can be approximated by $\pi(s, a) \approx \pi(s, a, \theta)$, where θ represents the weights of a neural network.

Among the various methods used to train neural networks for Reinforcement Learning, the Actor-Critic networks are used to learn simultaneously the policy and value functions. While the policy was presented in section III-A, the value function represents the expectation of how much reward to get in the future, and can be written as the Bellman's Equation:

$$V(s_t) = \mathbb{E}(r_t + \gamma V(s_{t+1})) \tag{3}$$

In the Actor-Critic methods, there are two neural networks: the actor network is policy-based, used to predict the best action given the current state, and the critic network is value-based, responsible for criticizing the policy based on the estimation of the value function, and is used only during the training process.

C. Central Pattern Generators

In biology, the Central Pattern Generator is a distributed neural network located on the spinal cord responsible for generating coordinated and periodic movements. In quadruped robotics, CPG oscillators are commonly used to generate rhythmic movements to control the joints of the robot [16], [17] or even to generate gait trajectories for the feet.

In this paper, we focus on the phase oscillator with controlled amplitude used in [18] to produce the locomotion behaviors of a salamander robot. This oscillator is widely used in many other works related to gait generation for quadruped robots [4], [12], [13].

$$\ddot{m}_i = c\left(\frac{c}{4}(\mu_i - m_i) - \dot{m}_i\right) \tag{4}$$

$$\dot{\theta}_i = \omega_i + \sum_j m_j w_{ij} \sin(\theta_j - \theta_i - \phi_{ij}) \tag{5}$$

In the equations (4) and (5), c is the convergence factor of the oscillator, θ_i and m_i represent the phase of the oscillator and its amplitude, ω_i and μ_i represent its intrinsic frequency and intrinsic amplitude, and the coupling between the oscillators are defined by the weights w_{ij} and biases ϕ_{ij}. In a trot gait, in which the diagonal feet are in a synchronous movement, the coupling matrix Φ can be defined as [12]:

$$\Phi = \begin{bmatrix} 0 & \pi & \pi & 0 \\ -\pi & 0 & 0 & -\pi \\ -\pi & 0 & 0 & -\pi \\ 0 & \pi & \pi & 0 \end{bmatrix} \tag{6}$$

IV. Development

A. Environment Creation

The MDP environment was created in Isaac Lab, a modular framework for robot learning powered by NVIDIA Isaac Sim [19] with a lot of useful tasks, reward and randomization functions, wrappers, algorithms, robotic platforms and so on.

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 07,2026 at 09:24:23 UTC from IEEE Xplore. Restrictions apply.

We used Unitree Go2 quadruped robot in this work, a small-size quadruped robot with 12 degrees of freedom. We train the robot on a flat ground environment, so we can easily see if there is any difference in the use of DRL in terms of energy consumption, and also we do not require any complex control system to be implemented on the quadruped to compare with.

The following topics detail the creation of the environment for training the agent.

1) Action Space: The action space is formed by the twelve desired joint position offsets, $\mathcal{A} = [q_1^*, q_2^*, q_3^*, \ldots, q_{12}^*] \in \mathbb{R}^{12}$, so that $a_t = q_j^*$. A Proportional-Derivative (PD) controller then produces motor torques to apply in the robot joints, as described in Section IV-B.

2) Observation Space: The observation space $\mathcal{O} \in \mathbb{R}^{52}$ consists of the input velocity command, proprioceptive measurements of the robot and the last action computed by the policy:

- $(v_{b,xy}^*, \omega_{b,z}^*) \in \mathbb{R}^3$ - the desired velocity commands;
- $v_b \in \mathbb{R}^3$ - the base linear velocity;
- $\omega_b \in \mathbb{R}^3$ - the base angular velocity;
- $p_b \in \mathbb{R}^3$ - the projected gravity;
- $q_j \in \mathbb{R}^{12}$ - the joint positions;
- $\dot{q_j} \in \mathbb{R}^{12}$ - the joint velocities;
- $f_c \in \mathbb{R}^4$ - the feet contact booleans;
- $a_{t-1} \in \mathbb{R}^{12}$ - the last action computed.

3) Reward Function: The reward function is composed of a series of reward and penalization terms to ensure that the quadruped can learn to locomote satisfactorily and with energy efficiency.

Rewards:

- $r_{v_{b,xy}}$ - rewards tracking of linear velocity commands;
- $r_{\omega_{b,z}}$ - rewards tracking of angular velocity commands;
- r_{air_time} - rewards long steps taken by the feet.

Penalizations:

- $r_{v_{b,z}}$ - penalizes z-axis base linear velocity;
- $r_{\omega_{b,xy}}$ - penalizes xy-axis base angular velocity;
- r_{action_rate} - penalizes the rate of change of the actions;
- r_{accel} - penalizes joint accelerations on the articulation;
- r_{power} - penalizes joint powers on the articulation.

Each one of these terms is multiplied by a weight, so the final reward function is given by:

$$r = a_1 r_{v_{b,xy}} + a_2 r_{\omega_{b,z}} + a_3 r_{air_time} + a_4 r_{v_{b,z}} + a_5 r_{\omega_{b,xy}} + a_6 r_{action_rate} + a_7 r_{accel} + a_8 r_{power} \quad (7)$$

The computation of each term, alongside its weights, is described in the table I, where ϕ is defined as $\phi(x) := \exp(-\frac{\|x\|^2}{0.25})$, $t_{air,f}$ is the time that the foot f is in the air, $q_{j,t}^*$ is the action taken by the policy at time t, τ_j are the joint torques and dt is the environment time step. These reward term definitions are similar to the ones used in [8].

Note that the joint power penalization is given by the sum of the instantaneous power of each motor of the robot, by multiplying their torques and velocities, instead of considering the energy consumption over a period ($E = \sum_{j=1}^{12} \int_0^T |\tau_j . \dot{q_j}| dt$). This is a similar approach done by other works [4], [9], [11], [20], is easiest to implement and can provide rewards at every time step, avoiding the sparse rewards problem [21].

TABLE I: Definition and weights of reward terms

Reward Term	Definition	Weight
$r_{v_{b,xy}}$	$\phi(v_{b,xy}^* - v_{b,xy})$	$2dt$
$r_{\omega_{b,z}}$	$\phi(\omega_{b,z}^* - \omega_{b,z})$	$1.5dt$
$r_{\text{air_time}}$	$\sum_{f=1}^{4}(t_{\text{air},f} - 0.5)$	$0.5dt$
$r_{v_{b,z}}$	$-v_{b,z}^2$	$1.0dt$
$r_{\omega_{b,xy}}$	$-\|\omega_{b,xy}\|^2$	$0.05dt$
$r_{\text{action_rate}}$	$-\|q_{j,t}^* - q_{j,t-1}^*\|^2$	$0.01dt$
r_{accel}	$-\|\ddot{q}_j\|^2$	$1.0 \times 10^{-6}dt$
r_{power}	$-\sum_{j=1}^{12} \|\tau_j \cdot \dot{q}_j\|$	$0.003dt$

TABLE II: Noises applied to the observations

Observation Term	σ
$v_{b,xy}$	0.05 m/s
$\omega_{b,z}$	0.05 rad/s
p_b	0.025 rad/s²
q_j	0.01 rad
$\dot{q_j}$	0.5 rad/s

4) Dynamic and Domain Randomization: To ensure that the robot can locomote with a continuous range of velocities and react to external disturbances and noises, we add some randomizations to the training process:

- The base mass of the robot is increased/decreased in a range of $[-1, 2]$ kg when each environment is created;
- The velocity commands $(v_{b,xy}^*, \omega_{b,z}^*)$ are resampled every 10 seconds, following a uniform distribution with range $[-1.0, 1.0]$ m/s for linear velocities and $[-\frac{\pi}{2}, \frac{\pi}{2}]$ rad/s for angular velocity.
- The robot suffers an external pushing by setting its base linear velocity to a value in a range of $[-0.5, 0.5]$ m/s for x and y axes, 10 to 15 seconds after the episode started.

Also, we add Gaussian noises to all the observations that provide information about the robot body and joint states, to simulate real-world sensors. All the inserted noises follow a normal distribution, centered on zero and with standard deviations σ given in table II.

5) Termination: The episode is terminated (due to a task failure) when the base of the robot touches the ground or when its orientation in roll/pitch exceeds the limit angle of $\frac{\pi}{2}$ rad, and the episode is truncated (due to a time-limit) when the maximum episode length of 20 seconds is reached.

B. Trainning and Control Framework

Fig. 1 shows the framework used for training the agent and controlling the robot. As explained in section III-B, the Critic Network is used only during the training process to help the agent learn the optimal policy. After the training, the Actor Network is used to receive feedback on the robot's states and control the robot to follow the desired velocity command by sending joint position offset commands. These joint position commands are converted to torques by a PD controller that computes the outputs as $\tau_j^* = k_p(q_j^* - q_j) + k_d(-\dot{q_j})$. We use $k_p = 25$ and $k_d = 0.5$. The simulation runs at 200 Hz, setting the control frequency to that value.

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 07,2026 at 09:24:23 UTC from IEEE Xplore. Restrictions apply.

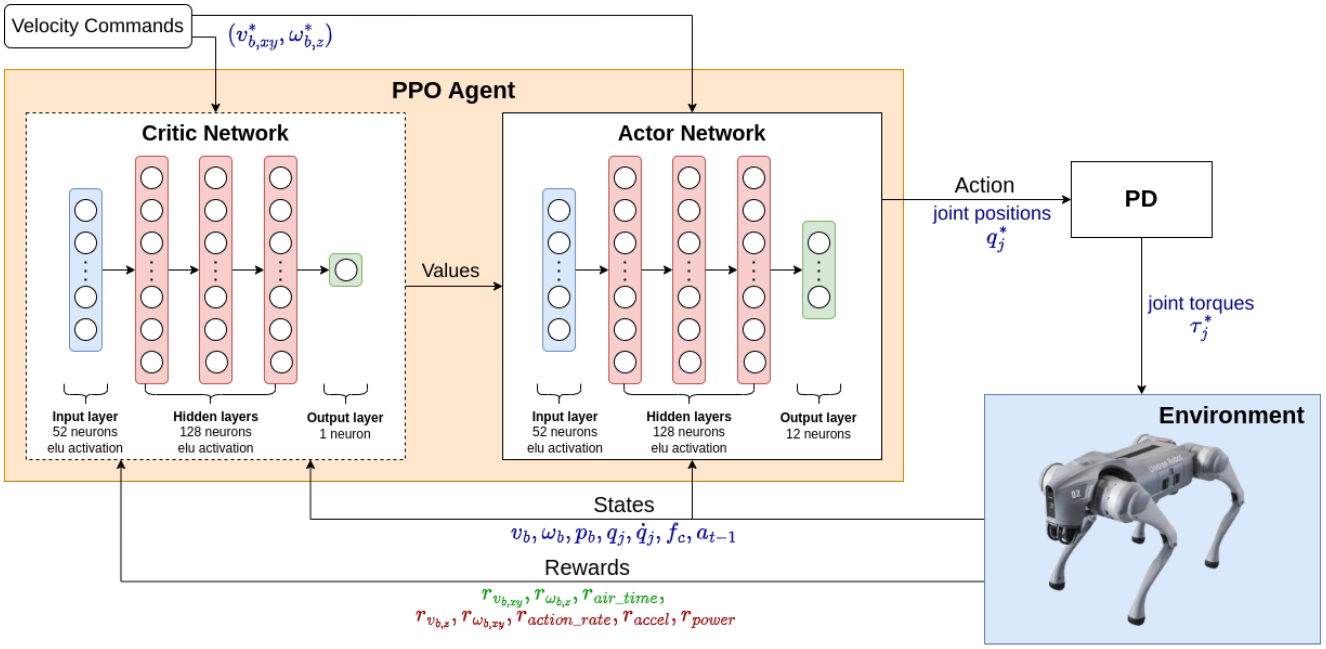


Fig. 1: Training framework and control diagram. Both actor and critic networks have an input layer with 52 neurons (the states, the last action and the velocity commands), 3 hidden layers with 128 neurons, and an elu activation function. While the critic outputs the value of the current state to help train the actor network, the latter outputs the twelve desired joint position offsets, to be sent to a PD controller that applies torques to the robot joints.

After modeling the environment and the Isaac Lab task, we trained the agent using the PPO [22] algorithm from the RSL-RL library [23]. We trained 2048 robots in parallel in a total of 20000 learning iterations, and the PPO hyper-parameters used for training are shown in table III. The training was performed in a notebook with an NVIDIA GeForce RTX 3060 GPU, Intel® Core™ i7-11800H processor and 16 GiB memory, and took approximately six hours to finish.

C. CPG Implementation

The CPG is used in this work to generate gait trajectories for each foot of the robot, in order to implement an open-loop control to compare its energy consumption with the trained controller. For that, we use the equations (4) and (5) to compute the phase and amplitude of the four oscillators (one per leg) at each time step. We set $c = 50$, $\mu_i = 1$, $w_{ij} = 1$ for any (i, j), and the coupling biases ϕ_{ij} can be obtained from (6). The intrinsic frequency of the oscillators is defined as $\omega_{swing} = 6.4\pi$ rad/s during the swing phase $(\sin\theta_i \geq 0)$ and $\omega_{stance} = 4.6\pi$ rad/s during the stance phase $(\sin\theta_i < 0)$.

TABLE III: Hyperparameters used for PPO training

Hyperparameter	Value
Batch size	49,152 (2,048 × 24)
Mini-batch size	8,192 (2,048 × 4)
Number of epochs	5
Clip range	0.2
Entropy coefficient	0.01
Discount factor (γ)	0.99
GAE discount factor (λ)	0.95
Desired KL-divergence	0.01
Learning rate	1×10^{-3} (Adaptive)

The phases and amplitudes of the oscillators are then used to compute 2D points in space:

$$x_{i,foot} = -d_{step} m_i \cos(\theta_i) \tag{8}$$

$$z_{i,foot} = \begin{cases} g_c \sin\theta_i, & \text{if } \sin\theta_i > 0 \\ g_p \sin\theta_i, & \text{otherwise} \end{cases} \tag{9}$$

where g_c is the ground clearance (maximum height of the foot during the swing phase), g_p is the ground penetration (maximum penetration on the ground during the stance phase), and d_{step} is the length of the step. We set $g_c = 0.05$ m, $g_p = 0.003$ m, and $d_{step} = 0.1$ m. The $(x_{i,foot}, z_{i,foot})$ points are then passed to inverse kinematics, so it can compute the joint angles to achieve the desired point in space, w.r.t the default position of each foot.

To control the robot using CPG, we also needed to increase the PD parameters to $k_p = 100$ and $k_d = 1$, to ensure that the joints can achieve the desired angular position more quickly: without this, the joints are not able to finish their movements and the robot simply cannot locomote.

V. Results

A. Trainning Results

Fig. 2 shows each one of the reward terms presented in section IV-A during the training process. After 2500 iterations the agent learned how to walk and follow a velocity command (Fig. 2a and 2b), penalizing non-desired velocities (Fig. 2d

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 07,2026 at 09:24:23 UTC from IEEE Xplore. Restrictions apply.

and 2e), and after 5000 iterations it learned how to do it more smoothly and with less energy consumption (Fig. 2g and 2h).

It is also interesting to know that, even without specifying during the training process, we observed that the agent learned to walk according to a trot gait (where the diagonal legs move synchronously), which can be more energy-efficient for some velocity commands [11], [24]. This can be seen in Fig. 3, where FL, FR, RL and RR represent the Front Left, Front Right, Rear Left and Rear Right feet contact booleans, filtered by a moving average with a window size of 10 steps to remove noises. It is noticeable that the diagonal feet (Front Right and Rear Left, for example) are in contact with the ground approximately at the same time (executing the stance phase), while the other feet are in the air (executing the swing phase).

B. Energy Consumption Analysis

The CPG parameters presented in section IV-C result in a linear velocity of approximately 1.0 m/s. Fig. 4 shows the total power of all twelve articulations ($P = \sum_{j=1}^{12} |\tau_j . \dot{q_j}|$),

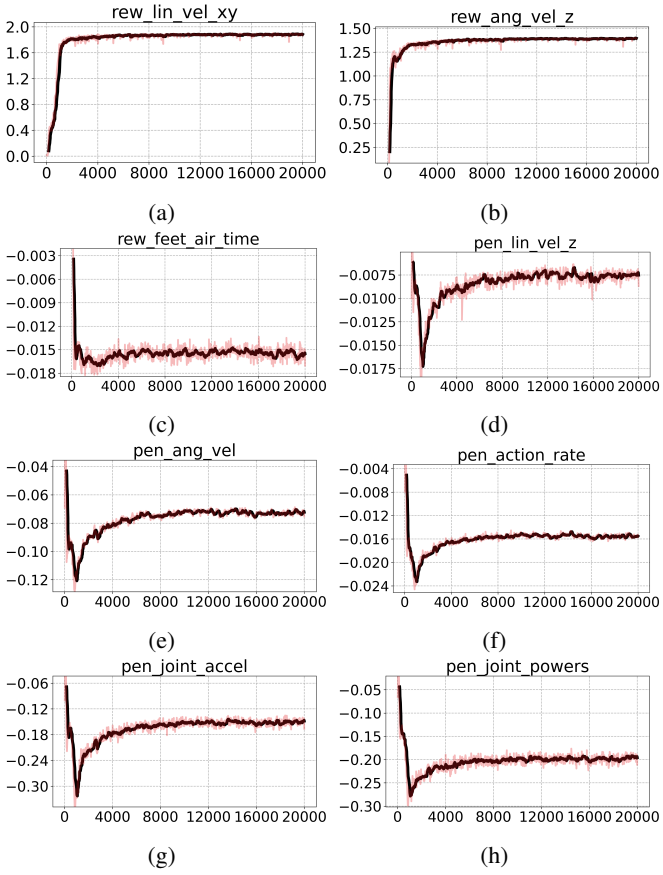


Fig. 2: Rewards and penalizations of each term during training. (a) Linear velocity xy tracking reward $a_1 r_{v_{b,xy}}$. (b) Angular velocity z tracking reward $a_2 r_{\omega_{b,z}}$. (c) Feet air time reward $a_3 r_{air_time}$. (d) Linear velocity z penalization $a_4 r_{v_{b,z}}$. (e) Angular velocity xy penalization $a_5 r_{\omega_{b,xy}}$. (f) Action rate penalization $a_6 r_{action_rate}$. (g) Joint acceleration penalization $a_7 r_{accel}$. (h) Joint power penalization $a_8 r_{power}$.

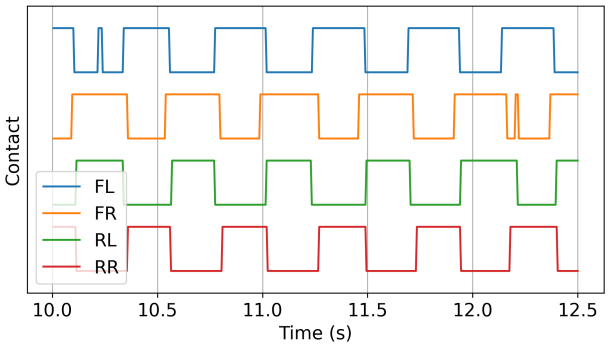


Fig. 3: Gait learned by the policy.

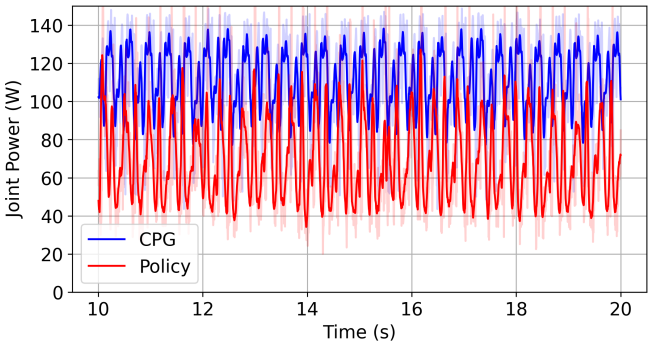


Fig. 4: Power consumption comparison for CPG and policy.

when following this velocity command, between $t = 10s$ and $t = 20s$. Despite greater oscillation, the controller using the trained policy spends significantly less energy compared to the CPG controller: an average instantaneous power of 111.2 W in CPG-based controller against 69.6 W in the trained policy-based controller, a reduction of approximately 37.4%.

We also computed the cost of transport (CoT), which considers the power consumed per unit mass and unit speed, CoT $= \frac{P}{mgv}$ [2]. Considering $m = 15$ kg, $g = 9.81$ m/s² and P and v are the average of power and the average of velocity, we have a CoT of 0.749 for CPG-based controller and 0.473 for policy-based controller. This experiment was replicated with different parameters for the CPG controller and different velocity commands for the policy, and the results can be seen in the table IV.

The results show that in nine of the ten experiments the trained policy requires less power to locomote at approximately the same velocity, and in eight of them it has a lower cost of transport. In the best case (first experiment, with the lower velocity command), the policy can spend up to 55.8% less energy when compared to the CPG controller, and the trained agent power consumption increases/decreases as the velocity commands also increase/decrease. It can also be seen that the robot can locomote with a velocity much bigger than the maximum 1 m/s velocity command used during training — a value 120% bigger in the last experiment —, and the only situations where the trained agent spent more energy (or

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 07,2026 at 09:24:23 UTC from IEEE Xplore. Restrictions apply.

TABLE IV: Comparison between CPG and the trained policy in terms of power consumption.

CPG Parameters					CPG			Policy		
ω_{swing}	ω_{stance}	g_c	g_p	d_{step}	Velocity (m/s)	Power (W)	CoT	Velocity (m/s)	Power (W)	CoT
4.8π	2π	0.05	0.002	0.08	0.365	71.109	1.324	0.365	**31.445**	**0.585**
5π	3π	0.05	0.005	0.10	0.582	80.352	0.938	0.596	**42.496**	**0.485**
9.2π	7.6π	0.03	0.001	0.06	0.744	74.922	0.684	0.736	**49.651**	**0.459**
5π	3π	0.05	0.004	0.14	0.876	124.097	0.963	0.878	**60.636**	**0.469**
6.4π	4.6π	0.05	0.003	0.10	1.009	111.223	0.749	1.000	**69.648**	**0.473**
7.2π	4.4π	0.04	0.002	0.12	1.250	138.451	0.760	1.250	**97.304**	**0.529**
8π	7π	0.04	0.002	0.09	1.415	127.636	0.613	1.399	**118.411**	**0.575**
10π	8π	0.03	0.005	0.09	1.685	**153.377**	**0.619**	1.677	166.401	0.674
9π	7π	0.05	0.004	0.12	2.090	241.125	**0.784**	1.999	**236.341**	0.803
12π	8π	0.07	0.002	0.12	2.411	396.713	1.118	2.280	**308.892**	**0.920**

have a greater CoT) are in the experiments with a velocity command not seen during training.

VI. Conclusion

In this work, we developed a locomotion controller for a quadruped robot based on Deep Reinforcement Learning to consume less energy. We use a reward function term that penalizes the instantaneous power consumption of the articulations, leading the robot to learn to walk energy-efficiently.

The results show that the trained policy can spend significantly less energy compared to CPG controller. The agent learned an efficient gait to locomote, the trot gait, even without specifying during training, and was also able to walk in velocities much bigger them those seen during training, maintaining a lower power consumption in most cases.

In addition to energy efficiency, the use of a control method based on Reinforcement Learning can also bring several other advantages, such as resilience to external disturbances, adaptability in unseen situations, simplicity, and little or no dependency on control parameters tuning, as required in CPG and many other control methods.

This project is open-source, was developed as an extension of Isaac Lab, and can be found in [25]. In further works, we will also use exteroceptive sensory data and curriculum learning to train the robot to walk on uneven terrain and stairs, and possibly with different gaits, allowing the robot to move in an energy-efficient manner over more complex terrain.

References

[1] L. Souza, F. Mohr, and B. Alencar, "Analysis, prototyping and locomotion control of a quadruped robot." IEEE, 10 2023, pp. 129–134. [Online]. Available: https://ieeexplore.ieee.org/document/10333039/

[2] G. Wang, L. Ding, H. Gao, Z. Deng, Z. Liu, and H. Yu, "Minimizing the energy consumption for a hexapod robot based on optimal force distribution," *IEEE Access*, vol. 8, pp. 5393–5406, 2020.

[3] Z. Yan, H. Ji, and Q. Chang, "Energy consumption minimization of quadruped robot based on reinforcement learning of ddpg algorithm," *Actuators*, vol. 13, 1 2024.

[4] G. Bellegarda and A. Ijspeert, "Cpg-rl: Learning central pattern generators for quadruped locomotion," *IEEE Robotics and Automation Letters*, vol. 7, pp. 12 547–12 554, 10 2022.

[5] Sustainable development goals. United Nations. [Online]. Available: https://sdgs.un.org/goals

[6] Z. Zhang, X. Chang, H. Ma, H. An, and L. Lang, "Model predictive control of quadruped robot based on reinforcement learning," *Applied Sciences (Switzerland)*, vol. 13, 1 2023.

[7] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science Robotics*, vol. 5, pp. 1–22, 2020.

[8] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," pp. 1–14, 2021. [Online]. Available: http://arxiv.org/abs/2109.11978

[9] M. Aractingi, P.-A. Léziart, T. Flayols, J. Perez, T. Silander, P. Souères, and P. Soueres, "Controlling the solo12 quadruped robot with deep reinforcement learning," 2022. [Online]. Available: https://github.com/Gepetto/soloRL

[10] S. Chen, B. Zhang, M. W. Mueller, A. Rai, and K. Sreenath, "Learning torque control for quadrupedal locomotion," 3 2022. [Online]. Available: http://arxiv.org/abs/2203.05194

[11] Z. Fu, A. Kumar, J. Malik, and D. Pathak, "Minimizing energy consumption leads to the emergence of gaits in legged robots." [Online]. Available: https://energy-locomotion.github.io.

[12] G. Bellegarda and A. J. Ijspeert, "Visual cpg-rl: Learning central pattern generators for visually-guided quadruped navigation," *ArXiv*, vol. abs/2212.14400, 2022. [Online]. Available: https://api.semanticscholar.org/CorpusID:255341202

[13] M. Shafiee, G. Bellegarda, and A. Ijspeert, "Puppeteer and marionette: Learning anticipatory quadrupedal locomotion based on interactions of a central pattern generator and supraspinal drive," vol. 2023-May. Institute of Electrical and Electronics Engineers Inc., 2023, pp. 1112–1119.

[14] R. S. Sutton and A. G. Barto, *Reinforcement learning : an introduction*. MIT Press, 2018.

[15] S. L. Brunton and J. N. Kutz, "Data-driven science and engineering machine learning, dynamical systems, and control," 2021.

[16] J. Wang, C. Hu, and Y. Zhu, "Cpg-based hierarchical locomotion control for modular quadrupedal robots using deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 6, 10 2021.

[17] L. Righetti and A. J. Ijspeert, "Pattern generators with sensory feedback for the control of quadruped locomotion," 2008, pp. 819–824.

[18] A. J. Ijspeert, A. Crespi, D. Ryczko, and J. M. Cabelguen, "From swimming to walking with a salamander robot driven by a spinal cord model," *Science*, vol. 315, pp. 1416–1420, 3 2007.

[19] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, and A. Garg, "Orbit: A unified simulation framework for interactive robot learning environments," *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023.

[20] B. Hu, S. Shao, Z. Cao, Q. Xiao, Q. Li, and C. Ma, *Learning a Faster Locomotion Gait for a Quadruped Robot with Model-Free Deep Reinforcement Learning (ROBIO)*. IEEE, 2019.

[21] M. Dawood, N. Dengler, J. de Heuvel, and M. Bennewitz, "Handling sparse rewards in reinforcement learning using model predictive control," 10 2022. [Online]. Available: http://arxiv.org/abs/2210.01525

[22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 7 2017. [Online]. Available: http://arxiv.org/abs/1707.06347

[23] M. Mittal, N. Rudin, and Z. Ali, "Rsl_rl: Reinforcement learning for robotics," 2023. [Online]. Available: https://github.com/leggedrobotics/rsl_rl

[24] J. Lei, F. Wang, H. Yu, T. Wang, and P. Yuan, "Energy efficiency analysis of quadruped robot with trot gait and combined cycloid foot trajectory," *Chinese Journal of Mechanical Engineering (English Edition)*, vol. 27, pp. 138–145, 1 2014.

[25] F. Mohr, "Isaacorbit-quadruped-rl," 2024. [Online]. Available: https://github.com/felipemohr/IsaacOrbit-Quadruped-RL

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 07,2026 at 09:24:23 UTC from IEEE Xplore. Restrictions apply.